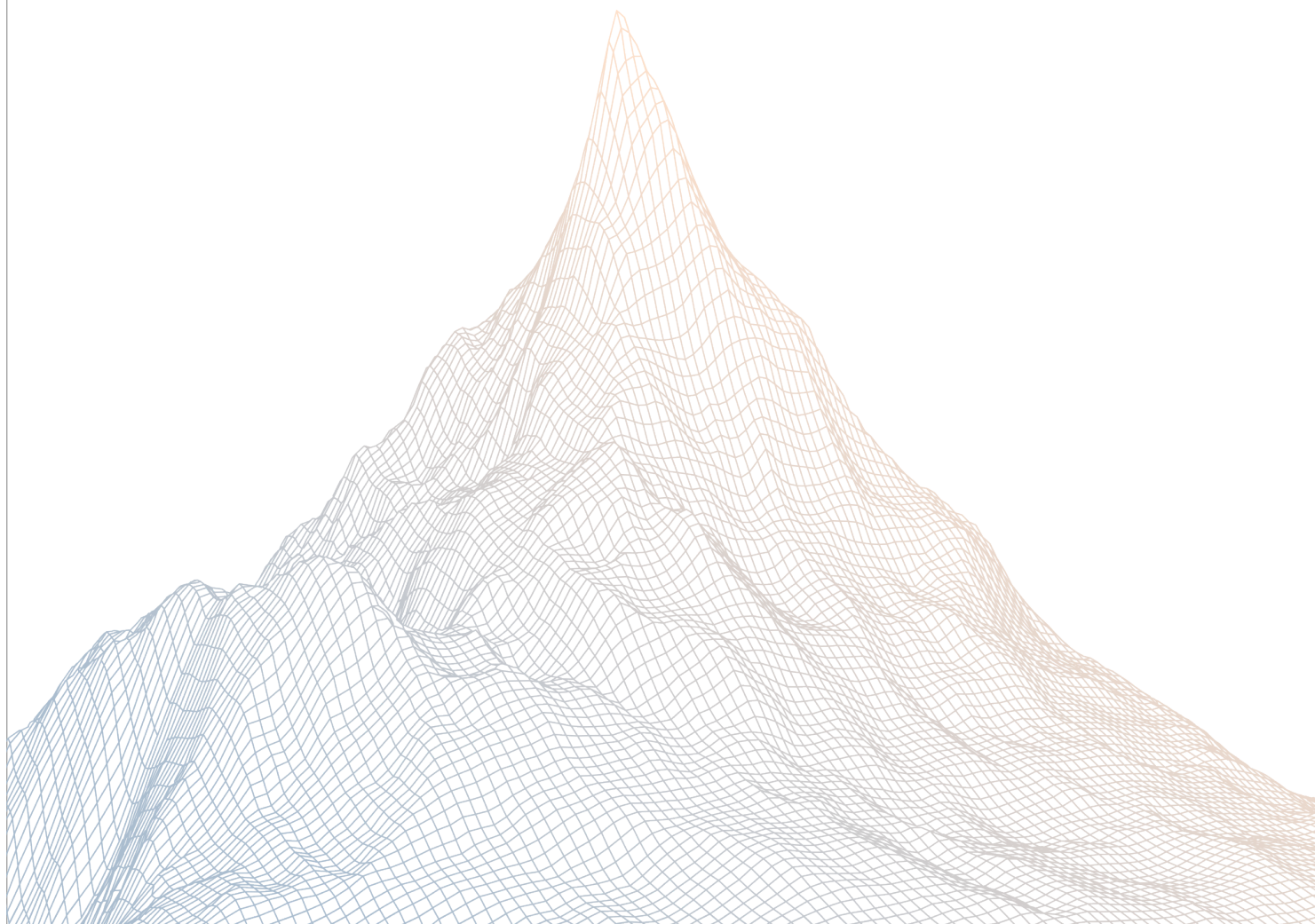


Dabba Network

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

March 26th to April 10th, 2026

AUDITED BY:

oakcobalt
ret2basic

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Dabba Network	4
2.2	Scope	4
2.3	Audit Timeline	7
2.4	Issues Found	7
3	Findings Summary	7
4	Findings	10
4.1	High	11
4.2	Medium	17
4.3	Low	34
4.4	Informational	57

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Dabba Network

The Dabba network is a DePIN marketplace designed to bring together all necessary stakeholders to deliver high speed internet access in the emerging markets, starting with India. The mission of the Dabba network is to bring a billion Indians internet access over the next decade. The Dabba Network uses the DBT token to incentivise its community and stakeholders to build the network.

2.2 Scope

The engagement involved a review of the following targets:

Target	token-mint-contract
Repository	https://github.com/wifidabba/token-mint-contract.git
Commit Hash	4ccb43a7e1bca2b21aa7e470364c4337cb321956
Files	programs/mint-playground/src/**/*.rs
Target	dabba-staking-contract
Repository	https://github.com/wifidabba/dabba-staking-contract
Commit Hash	3500edd466d4cc188b5a4efe845c3fd842227760
Files	programs/dabba_apy_staking/src/**/*.rs

Target	registry-contract
---------------	-------------------

Repository	https://github.com/wifidabba/registry-contract.git
-------------------	---

Commit Hash	305dc1dde105fd4cc18fd7c80d4cd60cd70654c7
--------------------	--

Files	programs/registry_contract/src/**/*.rs
--------------	--

Target	dabba-nft-contract
---------------	--------------------

Repository	https://github.com/wifidabba/dabba-nft-contract.git
-------------------	---

Commit Hash	66a13620b265871a260d52573c14bffcbeaf26d1
--------------------	--

Files	programs/dabba-nft-contract/src/**/*.rs
--------------	---

Target	Mitigation Review
---------------	-------------------

Repository	https://github.com/wifidabba/token-mint-contract.git
-------------------	---

Commit Hash	71d8f1cdc513e41d6f85200b45a45ab243d1ab24
--------------------	--

Files	programs/mint-playground/src/**/*.rs
--------------	--------------------------------------

Target	Mitigation Review
Repository	https://github.com/wifidabba/dabba-staking-contract
Commit Hash	2510a33d6d0efbbac51436af1f6c27d4203220c1
Files	programs/dabba_apy_staking/src/**
Target	Mitigation Review
Repository	https://github.com/wifidabba/registry-contract.git
Commit Hash	8b73e012748cf365f6b8561823536ba1ba869dd1
Files	programs/registry_contract/src/**/*.*rs
Target	Mitigation Review
Repository	https://github.com/wifidabba/dabba-nft-contract.git
Commit Hash	1447d6524c7893e06187389b0c59d860c4c7ee0d
Files	programs/dabba-nft-contract/src/**/*.*rs

2.3 Audit Timeline

March 26, 2026	Audit start
April 10, 2026	Audit end
April 11, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	3
Medium Risk	8
Low Risk	12
Informational	12
Total Issues	35

3

Findings Summary

ID	Description	Status
H-1	Incorrect root read from first 32 bytes of partner merkle tree account	Resolved
H-2	NFT authorization is fixed to a registry trust root that may not match the intended deployment	Resolved
H-3	Permissionless singleton bootstrap allows first-caller control capture across multiple programs	Resolved
M-1	Transferable hotspot-owner NFTs can strand registry-bound lifecycle rights	Resolved
M-2	Pre-mint can be blocked by pre-initializing canonical treasury ATAs	Resolved
M-3	Missing WalletPage role check in verify_hotspot_owner enables cross-role registry misuse	Resolved
M-4	Token mint hardcodes SPL-token ATA derivation despite token-interface support	Resolved
M-5	Staking does not validate external mint control features	Resolved
M-6	verify_partner_cnft_collection drops Bubblegum proof accounts	Resolved
M-7	Incorrect rpt accumulator and reward distribution in edge cases	Resolved
M-8	Core NFT attributes stay stale when hotspot owner PDA state changes	Resolved
L-1	Single-step super admin transfer contradicts stated two-step invariant	Resolved
L-2	No on-chain uniqueness check allows duplicate pubkeys per role registry	Acknowledged
L-3	daily_mint catch-up loop can permanently brick emissions	Resolved
L-4	New device NFT owner can delete prior-owner-linked DeviceAccount without owner continuity check	Acknowledged
L-5	User can prematurely burn and close hotspot owner account after protocol-funded mint	Acknowledged

ID	Description	Status
L-6	Update device field instructions accept unconstrained registry indices, allowing minter to silently re-attribute devices to arbitrary parties	Resolved
L-7	Staking reward accounting uses u64 and can overflow	Resolved
L-8	daily_mint is reachable before bootstrap premint completes	Resolved
L-9	Snapshot-based reward distribution diverges from documented Synthetix rewardPerToken accumulator model	Acknowledged
L-10	mint_partner_cnft does not pin partner_collection to admin_config.partner_collection_mint	Resolved
L-11	Stale dabba_nft_index persists after unstake	Resolved
L-12	Init allows Pubkey::default() for super admin and role authority while transfers reject it	Resolved
I-1	Pause scope is inconsistent across docs; Emergency pause omits super-admin rotation	Resolved
I-2	Incorrect error for reward-vault init ordering failure	Resolved
I-3	Unused StakingError variants LockExpired and Position-Cooling	Resolved
I-4	Unused token metadata constants DBT_DECIMALS and DBT_UNIT	Resolved
I-5	Manual stake allows LockPeriod::Genesis, diverging from 6m/12m-only intent	Resolved
I-6	Unused recompute_weight	Resolved
I-7	Insufficient validations	Resolved
I-8	Hotspot-owner registry slots are not uniquely bound to assets	Acknowledged
I-9	Delayed daily-mint backlog always follows the latest configured reward treasury	Acknowledged

ID	Description	Status
I-10	Initialize allows default role addresses for critical authorities	Resolved
I-11	Unused TooManyYearsPassed error variant creates dead error path ambiguity	Resolved
I-12	Redundant treasury_ata check after writing state.treasury_ata	Resolved

4

Findings

4.1 High

A total of 3 High findings were identified.

[H-1] Incorrect root read from first 32 bytes of partner merkle tree account

SEVERITY: High

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: High

Target

- [programs/dabba-nft-contract/src/instructions/verify_partner_collection.rs#L137](#)

Description:

verify_partner_collection treats the first 32 bytes of the merkle tree account as the concurrent merkle root passed to Bubblegum VerifyCollection. SPL spl-account-compression stores a fixed header (including type and configuration) before the serialized ConcurrentMerkleTree body.

```
// programs/dabba-nft-contract/src/instructions/verify_partner_collection.rs
// Extract root from merkle tree data on-chain (same as device
// verification)
let root_bytes: [u8; 32] = {
    let merkle_tree_data = ctx.accounts.merkle_tree.try_borrow_data()?;
    require!(
        merkle_tree_data.len() >= 32,
        DabbaNetworkError::MerkleTreeDataTooShort
    );
    merkle_tree_data[0..32]
        .try_into()
        .map_err(|_| DabbaNetworkError::MerkleTreeParseError)?
};
...
.root(root_bytes)
```

Impacts: Incorrectly parsed root_bytes will cause partner collection merkle tree verification to fail, effectively DoS verify_partner_collection flow.

Recommendations:

1. Consider aligning with `verify_device_collection`: add `root_bytes: [u8; 32]` to the program instruction and ABI, and pass that into `.root(root_bytes)`. For example:

```
// programs/dabba-nft-contract/src/lib.rs
pub fn verify_partner_cnft_collection(
    ctx: Context<VerifyPartnerCollection>,
    name: String,
    symbol: String,
    uri: String,
    leaf_index: u32,
    root_bytes: [u8; 32], // @audit align with verify_device_cnft_collection
) -> Result<()> {
    instructions::verify_partner_collection::handler(
        ctx, name, symbol, uri, leaf_index, root_bytes,
    )
}
```

2. Alternatively, implement a correct on-chain read of the current root from the concurrent merkle account layout (header size + tree deserialization) if on-chain derivation is preferred; add tests against a known tree account snapshot.

Dabba Network: Resolved with [PR-4](#). Changed `root_bytes` to be passed as an instruction parameter instead of reading from the first 32 bytes of the merkle tree account data. This aligns with the `verify_device_collection` approach.

Zenith: Verified.

[H-2] NFT authorization is fixed to a registry trust root that may not match the intended deployment

SEVERITY: High**IMPACT:** High**STATUS:** Resolved**LIKELIHOOD:** Medium

Target

- [dabba-nft-contract/src/registry_cpi.rs](#)
- [dabba-nft-contract/src/instructions/init_and_mint_hotspot_owner.rs](#)
- [dabba-nft-contract/src/instructions/mint_partner_cnft.rs](#)
- [dabba-nft-contract/src/instructions/verify_device_collection.rs](#)
- [wifidabba-registry-contract/programs/registry_contract/src/lib.rs](#)

Description:

The NFT program is permanently bound to a single registry program ID through `get_registry_program_id()`, which returns `EqrCmUS4TXPVoKzXESxBMwabhroShSasuDmZxyP74XE2`. Every registry-gated authorization path, including `verify_hotspot_owner`, `verify_registry_index`, and `verify_registry_index_owner`, rejects accounts not owned by that exact program.

The co-located registry package in this repository declares a different program ID, `F4FU8VMGkxtxtfQ7JwSrHQNb7MMzmoQKNRw4aWdjCWv`. If these two repos are intended to be deployed together, that mismatch means the NFT program will not treat the co-deployed registry as authoritative. Hotspot-owner, partner, and device authorization will either fail against the local registry accounts or resolve against some other registry deployed at the hardcoded ID on the same cluster.

The core issue is not simply that a value is hardcoded. The deeper problem is that the NFT program's registry trust root is fixed and not configurable per deployment. If the hardcoded ID is the wrong trust root for the target environment, the program cannot be pointed at the intended registry without an upgrade.

This finding is therefore conditional on deployment intent. If `EqrCmUS4 ...` is the canonical production registry by design, then the local registry repo is stale or environment-specific and the risk is mainly configuration drift. If the NFT deployment is supposed to trust the registry contract packaged alongside it, then the mismatch is a real authorization bug.

Recommendations:

If the NFT program is meant to support different deployments or a co-deployed registry, store the trusted registry program ID in `AdminConfig` during trusted initialization and validate all registry accounts and PDAs against that configured value.

If a single global registry is intentional, eliminate source-of-truth drift by defining the registry ID once and sharing it across the NFT and registry packages, and document explicitly that the NFT program is not intended to trust the co-located registry artifact when the IDs differ.

Dabba Network: Resolved with [@cdd0d2dbb8 ...](#). Updated the hardcoded registry program ID in `registry_cpi.rs` to match the co-located registry contract's `declare_id` (`F4FU8VMGkxtextfQ7JwSrhQNb7MMzmoQKNRw4aWdjCWv`). Comment updated to reflect the correct ID. The registry program ID is intentionally hardcoded — single global registry, not configurable per deployment.

Zenith: Verified.

[H-3] Permissionless singleton bootstrap allows first-caller control capture across multiple programs

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [wifidabba-registry-contract/programs/registry_contract/src/lib.rs](#)
- [wifidabba-registry-contract/migrations/deploy.ts](#)
- [dabba_apy_staking/src/instructions/initialize.rs](#)
- [dabba_apy_staking/src/instructions/initialize_vaults.rs](#)
- [wifidabba-dabba-staking-contract/migrations/deploy.ts](#)
- [wifidabba-token-mint-contract/programs/mint-playground/src/lib.rs](#)
- [wifidabba-token-mint-contract/migrations/deploy.ts](#)

Description:

Three programs in this suite share the same root cause: a fixed singleton PDA is initialized by a public first-writer flow with no on-chain binding to a trusted deployment authority, while the in-repo deployment scripts do not perform an atomic protected bootstrap.

The pattern appears in the following places:

1. Registry program.

`initialize_global_config` creates the singleton `global_config` PDA at `['global-config']` and writes a caller-supplied `super_admin` into state. The initializer only requires a generic payer signer, so any first caller can capture registry governance and then control registry creation, pause, unpause, and authority transfer.

2. Staking program.

`initialize` creates the singleton `global_state` PDA at `['global_state']`, stores `super_admin` from the caller signer, and persists caller-chosen values for `reward_depositor`, `operator`, `pause_authority`, and `stake_mint`. After that first write, `initialize_vaults` and `initialize_reward_vault` authorize solely against the stored `global_state.super_admin`, so the same first caller can complete staking bootstrap and capture the custodial operator plane.

3. Token mint program.

`initialize_state_and_mint` creates the singleton state, mint, and mint_auth PDAs, makes the first caller `state.admin`, and accepts caller-controlled values for the config admin, daily mint admin, burn admin, treasury, daily reward treasury, and issuance schedule. That captured bootstrap then governs `pre_mint_tokens` and `daily_mint`, letting the attacker control premint destination and future emissions.

This is not just a documentation concern. All three repositories ship with effectively empty migrations/`deploy.ts` scripts, and anchor `deploy` does not itself initialize program state. That leaves a real public first-writer window unless bootstrap is handled externally and atomically.

Recommendations:

Treat singleton bootstrap as privileged initialization everywhere it appears. Bind each initializer to a trusted deployment authority, ideally the program upgrade authority or another immutable deployer key, and collapse follow-on bootstrap steps into a protected release flow so there is no exposed first-writer window on fixed PDAs. If operational atomic bootstrap is still required, document it as a hard deployment invariant and enforce as much of it on-chain as possible.

Dabba Network: Resolved with [PR-5](#), [PR-4](#) and [PR-1](#). All three program initializers now verify that the caller is the program's upgrade authority by reading the `BPFLoaderUpgradeable ProgramData` account and checking `upgrade_authority_address == signer`. Only the deployer can call `initialize`.

Zenith: Verified.

4.2 Medium

A total of 8 Medium findings were identified.

[M-1] Transferable hotspot-owner NFTs can strand registry-bound lifecycle rights

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [dabba-nft-contract/src/instructions/init_and_mint_hotspot_owner.rs](#)
- [dabba-nft-contract/src/instructions/stake.rs](#)
- [dabba-nft-contract/src/instructions/unstake.rs](#)
- [dabba-nft-contract/src/instructions/close_hotspot_owner_account.rs](#)

Description:

Hotspot-owner Core NFTs are minted directly to the user with only an Attributes plugin, while the on-chain lifecycle state stores an immutable `ho_address_index` taken from the registry at mint time. Later lifecycle actions still require both live asset ownership and that the acting wallet matches the registry wallet at that stored index.

Because the minted Core asset remains transferable while that registry binding stays fixed, a normal transfer or wallet migration can strand the lifecycle. A new owner can hold the NFT but fail registry-gated stake or close flows, while a staked NFT can only be released back to the registry-bound wallet rather than any later transferee.

Recommendations:

Choose one ownership model and enforce it consistently. Either make hotspot-owner assets non-transferable while registry binding is immutable, or add an authenticated rebind flow that updates the stored registry relationship when legitimate ownership changes.

Dabba Network: Resolved with [@7b59779ca7 ...](#)

Added `rebind_hotspot_owner` instruction (minter-only) that allows updating `ho_address_index` after an NFT transfer. The flow:

1. NFT is transferred to new owner (standard MPL Core transfer, only possible while unstaked)
2. Minter calls `rebind_hotspot_owner` which verifies the new owner holds the NFT and is registered in the registry, then updates the PDA's `ho_address_index`
3. New owner can now stake normally
4. The instruction requires: admin/minter signature, NFT must be unstaked, new owner must hold the NFT (verified by reading MPL Core asset owner field), and new owner must be registered in the `HOTSPOT_OWNER` registry.

Zenith: Verified.

[M-2] Pre-mint can be blocked by pre-initializing canonical treasury ATAs

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [programs/mint-playground/src/lib.rs#L823](#)
- [programs/mint-playground/src/lib.rs#L831](#)

Description:

After `initialize_state_and_mint`, `state.treasury`, `state.daily_reward_treasury`, and the mint PDA are public. Associated token account addresses for those wallets and the mint are therefore fully derivable.

Anyone may permissionlessly initialize those ATAs before the admin runs `pre_mint_tokens`. The `PreMintTokens` context uses Anchor `init` on both treasury ATAs, which attempts to allocate accounts that already exist and is expected to fail validation or CPI in that case, so the pre-mint instruction cannot be executed.

An example shape of the attack: They can calculate the address and initialize the ATA with the following code after the program is deployed before pre mint calls.

```
const [mintPda] = PublicKey.findProgramAddressSync(
  [Buffer.from("mint")],
  programId
);
const treasuryAta = await getAssociatedTokenAddress(mintPda, treasury);
const dailyRewardTreasuryAta = await getAssociatedTokenAddress(
  mintPda,
  dailyRewardTreasury
);
const tx = new Transaction().add(
  createAssociatedTokenAccountInstruction(
    attackerKeypair.publicKey,
    treasuryAta,
    treasury,
    mintPda
  ),

```

```

createAssociatedTokenAccountInstruction(
    attackerKeypair.publicKey,
    dailyRewardTreasuryAta,
    dailyRewardTreasury,
    mintPda
)
);
await sendAndConfirmTransaction(connection, tx, [attackerKeypair]);

```

Vulnerable code:

```

#[account(
    > init,
    payer = admin,
    associated_token::authority = treasury,
    associated_token::mint = mint,
)]
pub treasury_ata: Box<InterfaceAccount<'info, TokenAccount>>,

#[account(
    > init,
    payer = admin,
    associated_token::authority = daily_reward_treasury,
    associated_token::mint = mint,
)]
pub daily_reward_treasury_ata: Box<InterfaceAccount<'info,
TokenAccount>>,

```

Impacts: Dos of pre_mint_tokens: pre_mint_tokens can fail when canonical treasury ATAs already exist, blocking the intended one-shot pre-mint and ATA registration step.

Recommendations:

Consider changing both treasury ATA constraints in PreMintTokens from init to init_if_needed, with the same associated_token::authority, associated_token::mint, payer, and token_program alignment as today. For example,

```

#[account(
    init_if_needed,
    payer = admin,
    associated_token::authority = treasury,
    associated_token::mint = mint,
)]
pub treasury_ata: Box<InterfaceAccount<'info, TokenAccount>>,

```

```
#[account(  
  init_if_needed,  
  payer = admin,  
  associated_token::authority = daily_reward_treasury,  
  associated_token::mint = mint,  
)]  
pub daily_reward_treasury_ata: Box<InterfaceAccount<'info, TokenAccount>>,
```

Dabba Network: Resolved with [PR-3](#). Changed both treasury ATA constraints in PreMintTokens from init to init_if_needed. If ATAs are pre-initialized by a third party, the instruction now succeeds instead of failing.

Zenith: Verified.

[M-3] Missing WalletPage role check in verify_hotspot_owner enables cross-role registry misuse

SEVERITY: Medium

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/dabba-nft-contract/src/registry_cpi.rs#L116](#)

Description:

ho_address_index on HotspotOwnerAccount is defined as an index into the Hotspot Owner wallet registry (role 3). verify_hotspot_owner enforces that registry_config deserializes to HOTSPOT_OWNER_ROLE and that global_index is within registry.total_wallets, then locates (page_index, offset) and reads page.wallets[offset]. It checks page.page_index but never requires page.role == HOTSPOT_OWNER_ROLE.

Wallet pages are different PDAs per (role, page_index), so a caller can pass another role's wallet page that still matches the computed page_index and contains expected_owner at offset, while ho_address_index semantically refers to the hotspot registry only.

```
// programs/dabba-nft-contract/src/registry_cpi.rs
let page = WalletPage::deserialize(&mut page_slice)
    .map_err(|_| DabbaNetworkError::InvalidRegistryRole)?;

require!(
    page.page_index == page_index,
    DabbaNetworkError::InvalidRegistryRole
);

//@audit No check that page.role matches HOTSPOT_OWNER_ROLE before indexing
//wallets.
> let registry_owner = page.wallets[offset];

require!(
    registry_owner == *expected_owner,
    DabbaNetworkError::RegistryOwnershipMismatch
);
```

Callers: `init_and_mint_hotspot_owner`, `stake`, `unstake`, `close_hotspot_owner_account` (all pass unchecked `registry_config` / `wallet_page` `AccountInfo` and rely on this helper).

Impacts:

- `unstake` (worst): Only minter signs; user is not a signer. If `wallet_page` is a non-hotspot-owner page where `wallets[offset] = user` for a colliding registration, the instruction can transfer the NFT to user even when that pubkey is not the hotspot owner at `ho_address_index` on the hotspot registry—wrong-recipient / theft-class outcome relative to protocol intent.
- `init_and_mint_hotspot_owner`: Minter-controlled flow can bind a new HO NFT and PDA to registry proof that used a wrong role's row at the same (`page_index`, `offset`).
- Other caller path has lesser impact due to MPL Core ownership validation for transfer or burn.

Likelihood: In practice, incidence therefore hinges on minter behavior—honest. The threat model and risk depend on the tooling's runbook, or if the minter account is compromised or has incorrect account resolution.

Recommendations:

1. After deserializing `WalletPage` in `verify_hotspot_owner`, add `require!(page.role == HOTSPOT_OWNER_ROLE, DabbaNetworkError::InvalidRegistryRole)` (align with `verify_registry_index`). For example,

```
// programs/dabba-nft-contract/src/registry_cpi.rs
pub fn verify_hotspot_owner(
    registry_config: &AccountInfo,
    wallet_page: &AccountInfo,
    ho_address_index: u32,
    expected_owner: &Pubkey,
) -> Result<Pubkey> {
    ...
    require!(
        page.role == HOTSPOT_OWNER_ROLE,
        DabbaNetworkError::InvalidRegistryRole
    );
}
```

2. Add a negative test: hotspot `registry_config` + other-role `wallet_page` with matching `page_index` and `wallets[offset] = expected_owner` must fail.

Dabba Network: Resolved with [@5f9d0f7247 ...](#). Added `require!(page.role == HOTSPOT_OWNER_ROLE)` check in `verify_hotspot_owner` before reading `wallets[offset]`, preventing cross-role page substitution.

Zenith: Verified.

[M-4] Token mint hardcodes SPL-token ATA derivation despite token-interface support

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/mint-playground/src/lib.rs#L838-L844](#)

Description:

The token-mint program uses `InterfaceAccount<Mint>` and `Interface<TokenInterface>`, so the initialization path can be driven with either classic SPL Token or Token-2022.

However, the instruction logic later validates treasury ATAs with `get_associated_token_address`, which derives the classic SPL-token ATA address. Token-2022 mints use a different canonical ATA derivation because the token program ID is part of the seed set.

As a result, if the mint was created under Token-2022:

- `pre_mint_tokens` rejects the canonical treasury ATA
- `daily_mint` rejects the canonical reward ATA
- `update_daily_reward_treasury` stores and validates addresses against the wrong derivation rule

The program therefore advertises token-interface flexibility but hardcodes classic-token ATA assumptions in the actual control path.

Recommendations:

Either restrict the program to classic SPL Token explicitly, or derive ATAs with the active token program ID everywhere a mint/account relationship is validated.

Dabba Network: Resolved with [@fd231455af ...](#). Restrict token program to classic SPL Token: the program uses `InterfaceAccount` and `Interface`, which accept both SPL Token and Token-2022 at the type level. However, the instruction logic validates treasury ATAs via `get_associated_token_address`, which derives the classic SPL-token ATA address. Token-2022 mints use a different derivation (the token program ID is part of the seed set), so passing Token-2022 would cause `pre_mint_tokens`, `daily_mint`, and `update_daily_reward_treasury` to reject the canonical ATA.

Rather than reworking ATA derivation to support both programs, this change adds an explicit runtime constraint on every token_program account field that rejects Token-2022 (UnsupportedTokenProgram error). This makes the classic-only assumption enforceable on-chain rather than implicit.

Zenith: Verified.

[M-5] Staking does not validate external mint control features

SEVERITY: Medium

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/dabba_apy_staking/src/instructions/initialize.rs#L102](#)
- [programs/dabba_apy_staking/src/instructions/initialize_vaults.rs#L98](#)

Description:

The staking program accepts an arbitrary external `stake_mint` via `InterfaceAccount<Mint>` and never validates whether that mint carries privileged transfer or denial controls outside the staking program.

This exposes the vaults to mint-level features the program does not model, including:

- Token-2022 `PermanentDelegate`, which can transfer tokens from vault accounts without the PDA owner's signature
- non-`None` freeze authority on classic SPL or Token-2022 mints, which can freeze vault accounts externally
- Token-2022 transfer hooks, which can deny or selectively allow outbound vault transfers under external program control

The staking program's own access controls do not help once such a mint has been accepted, because the attack executes through the token program or hook program, not through staking instructions.

Recommendations:

Reject unsafe mint configurations at initialization. At minimum:

- require `freeze_authority = None`
- reject Token-2022 mints with `PermanentDelegate` or `TransferHook`
- consider restricting staking to the expected canonical mint instead of arbitrary token-interface mints

Dabba Network: Resolved with [@8d3d3eb2fe ...](#) Two validation checks added to initialize:

1. `token_program.key() == spl_token::ID` — Rejects Token-2022 mints entirely, eliminating

PermanentDelegate and TransferHook attack surfaces

2. `stake_mint.freeze_authority.is_none()` — Rejects any mint with a freeze authority set

Zenith: Verified.

[M-6] verify_partner_cnft_collection drops Bubblegum proof accounts

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [dabba-nft-contract/src/instructions/verify_partner_collection.rs](#)
- [dabba-nft-contract/src/instructions/verify_device_collection.rs](#)

Description:

Bubblegum `verify_collection` may require proof nodes that are not already cached in the tree account. In `mpl-bubblegum 1.4.0`, those extra proof nodes reach the CPI only through `VerifyCollectionCpiBuilder::add_remaining_account(s)`.

`verify_partner_cnft_collection` invokes `VerifyCollectionCpiBuilder` without iterating `ctx.remaining_accounts`, so any proof accounts supplied by the caller are silently dropped.

The sibling device-verification path already forwards every proof account before `invoke_signed`. The partner-verification path does not. For partner trees where Bubblegum still needs external proof nodes, partner collection verification fails even when the caller supplies the correct proof.

Recommendations:

Mirror the device flow and forward every proof account from `ctx.remaining_accounts` into `VerifyCollectionCpiBuilder` before `invoke_signed`. Add an integration test that uses a partner tree configuration requiring at least one external proof node.

Dabba Network: Resolved with [PR-4](#). Added proof node forwarding from `remaining_accounts` to the `VerifyCollectionCpiBuilder` in `verify_partner_collection`, mirroring the device verification path.

Zenith: Verified.

[M-7] Incorrect rpt accumulator and reward distribution in edge cases

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [dabba_apy_staking/src/instructions/update_daily_rewards.rs#L21-L25](#)

Description:

The staking contract uses a reward-per-token accumulator. When `update_daily_rewards` is called, it divides the deposited amount by `total_weighted_stake` at the moment of the call to produce a single `rpt_increment`. This is correct only when each call covers exactly one epoch and the pool composition has not changed since that epoch began. When the call covers multiple epochs (catch-up batch) or when new stakers joined during the covered period, the denominator is wrong for all but the most recent snapshot, producing two layers of error: a global rpt error and a per-user `reward_per_token_paid` baseline error.

```
let rpt_increment = (amount as u128)
    .checked_mul(PRECISION)
    .ok_or(StakingError::MathOverflow)?
    .checked_div(global.total_weighted_stake)
    .ok_or(StakingError::MathOverflow)?;
```

See the example below: `daily_mint_amount` = 1000 tokens, Alice and Carol stake the whole time. Bob joins on Day 3. Equal weights 500w each when all three are in the pool.

Case A: daily calls

- Alice: $500 + 500 + 333 = 1333$ tokens.
- Carol: $500 + 500 + 333 = 1333$ tokens.
- Bob: 333 tokens.

Case B: Days 1 and 2 missed; one deposit on Day 3

- Alice: $500w \times 2 = 1000$ tokens.
- Carol: $500w \times 2 = 1000$ tokens.
- Bob: $500w \times 2 = 1000$ tokens.

Likelihood context:

- Depositor offline: documented as real possibility and the intent is if offline: rewards accumulate but don't distribute until resumed.
- Pause/unpause: During a pause the pool can shrink via unstakes while `reward_per_token_stored` does not advance. After unpause, any catch-up deposit applies one `rpt_increment` over the full paused window using the post-unpause pool size, triggering the same mis-allocation.

Impacts:

- New stakers who joined while reward deposits were lagging receive rewards from epochs they were not part of. Users can game such a batch deposit period through front-running `update_daily_rewards`
- Stakers present during missed epochs receive less than their proportionally correct reward share; the deficit is permanently baked into global `rpt`.
- The loss scales with the number of missed epochs, the size of new stake entries.

Recommendations:

It's recommended to align staking reward accounting with per-epoch pool snapshots, and avoid recording a stale global `rpt` as user baseline. For example:

1. Consider introducing a fixed-size ring buffer of per-epoch `total_weighted_stake` snapshots

in `GlobalStakingState` (e.g. last 7 epochs). When distributing catch-up or post-pause rewards, apply each epoch's deposit against its recorded snapshot rather than the live pool size, ensuring the global `rpt` increment uses the correct denominator for each covered epoch.

2. Consider adding a `last_reward_epoch` field to `GlobalStakingState`. At `UserStakeAccount`

creation, if the last reward epoch is stale, defer `reward_per_token_paid` initialization and correct it via a post-backfill maintenance step.

Dabba Network: Resolved with [@bb9861a15a ...](#). Additionally, we plan to enforce backend checks to mitigate this scenario.

Zenith: Verified, and acknowledged the additional backend mitigation.

[M-8] Core NFT attributes stay stale when hotspot owner PDA state changes

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [dabba-nft-contract/src/instructions/init_and_mint_hotspot_owner.rs#L142-L152](#)

Description:

At mint, the Hotspot Owner Core asset is created with an MPL Core Attributes plugin (status, hotspot_nft). The program then updates HotspotOwnerAccount PDA fields on stake and unstake, but never issues an MPL Core plugin update. The on-chain Core account therefore retains the initial attribute values while the PDA (and events) reflect the true lifecycle. Two on-chain representations of “status” and linkage diverge after the first stake.

```
// programs/dabba-nft-  
contract/src/instructions/init_and_mint_hotspot_owner.rs  
let attributes_plugin = PluginAuthorityPair {  
  plugin: Plugin::Attributes(mpl_core::types::Attributes {  
    attribute_list: vec![  
      Attribute {  
        key: "status".to_string(),  
        value: "UNSTAKED".to_string(),  
      },  
      Attribute {  
        key: "hotspot_nft".to_string(),  
        value: "pending".to_string(),  
      },  
    ],  
  }),  
  authority: None,  
};  
...  
plugins: Some(vec![attributes_plugin]),
```



```
// programs/dabba-nft-contract/src/instructions/stake.rs
    hotspot_owner_account.status = HotspotOwnerAccount::STATUS_STAKED;
    hotspot_owner_account.staked_at = staked_at;
    hotspot_owner_account.dabba_nft_index = dabba_nft_index;
//@audit No UpdatePluginV1Cpi or other MPL Core call to align Attributes
with stake.
```

Impacts:

- On-chain inconsistency: Core Attributes can show UNSTAKED / pending while the PDA is STAKED and dabba_nft_index is set (and vice versa after unstake for status semantics).
- Any off-chain logic, reward logic derived from plugin state will be incorrect, potentially causing users' reward loss depending on off-chain business logic.

Recommendations:

1. Add UpdatePluginV1Cpi in stake and unstake (and any other instruction that changes Hotspot Owner lifecycle or linkage) so Attributes keys match the authoritative PDA fields and naming your product expects.
2. Add an integration or test that asserts parity between HotspotOwnerAccount and Core Attributes after mint, stake, and unstake

Dabba Network: Resolved with [@c8ff094b62 ...](#) and [@31e141856d ...](#)

Zenith: Verified.

4.3 Low

A total of 12 Low findings were identified.

[L-1] Single-step super admin transfer contradicts stated two-step invariant

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/registry_contract/src/lib.rs#L117](#)

Description:

The registry audit scope documents the invariant that super admin transfer is two-step (propose + accept-style flow). The program implements `transfer_super_admin` as a single signed instruction that immediately overwrites `GlobalConfig.super_admin`. There is no pending authority, acceptance by the incoming admin, cancel path, or timelock on-chain. This is a spec / safety-property gap: privileged mistake, malicious co-signer scenario, or compromised super-admin key yields immediate, irreversible control handoff with no on-chain challenge window.

```
// programs/registry_contract/src/lib.rs
pub fn transfer_super_admin(
    ctx: Context<TransferSuperAdmin>,
    new_super_admin: Pubkey,
) -> Result<> {
    let config = &mut ctx.accounts.global_config;
    require!(
        new_super_admin != Pubkey::default(),
        ErrorCode::InvalidAuthority
    );
    require!(
        config.super_admin == *ctx.accounts.super_admin.key,
        ErrorCode::Unauthorized
    );
    let old_super_admin = config.super_admin;
```

```
▷   config.super_admin = new_super_admin;
    ...
}
```

Impacts:

- Immediate loss of super-admin control (pause, registry initialization authority) to new_super_admin in one transaction; old admin cannot cancel on-chain after signing once.
- Fat-finger or wrong new_super_admin can lock governance if nobody holds the new key; malicious or coerced transfer cannot be rolled back by contract rules alone.
- Misaligns implementation with documented Registry invariant.

Recommendations:

Add an on-chain two-step flow: e.g. propose_super_admin sets pending_super_admin (+ optional propose_ts), accept_super_admin requires new_super_admin signer and clears pending; cancel_super_admin allows current super admin (or both parties per policy) to abort before acceptance. For example:

```
// programs/registry_contract/src/lib.rs
#[account]
pub struct GlobalConfig {
    pub super_admin: Pubkey,
    pub pending_super_admin: Option<Pubkey>, //@audit set by propose;
    cleared on accept/cancel
    pub paused: bool,
    pub bump: u8,
}
```

Dabba Network: Resolved with [PR-3](#). Replaced single-step transfer_super_admin with a two-step flow:

- propose_super_admin: current admin proposes new admin
- accept_super_admin: new admin signs to accept
- cancel_super_admin_transfer: current admin can cancel pending proposal
- Added pending_super_admin field to GlobalConfig and NoPendingTransfer error code.

Zenith: Verified.

[L-2] No on-chain uniqueness check allows duplicate pubkeys per role registry

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [programs/registry_contract/src/lib.rs#L237-L239](#)
- [programs/registry_contract/src/lib.rs#L318-L319](#)

Description:

One of the stated registry invariant: each wallet appears at most once per role (no duplicate registrations).

`append_wallet` and `append_wallets_batch` append pubkeys to paginated `WalletPage` vectors and increment `RegistryConfig.total_wallets` without checking whether that pubkey already exists for the role. The same role authority can therefore register the same wallet many times, or a buggy operator backend can do so. This is a specification—implementation gap rather than permissionless exploitation.

```
// programs/registry_contract/src/lib.rs
// Append wallet and update counts
page.wallets.push(wallet_pubkey);
page.count = page.count.saturating_add(1);
registry.total_wallets = registry.total_wallets.saturating_add(1);
```

```
// programs/registry_contract/src/lib.rs
// append_wallets_batch inner loop
page.wallets.push(wallet_pubkey);
page.count = page.count.saturating_add(1);
current_total = current_total.saturating_add(1);
```

Impacts:

- Documented invariant “at most once per role” is not enforced on-chain.
- `total_wallets` and global indices over-count if duplicates exist; consumers that assume a set may miscompute membership cardinality, analytics, or downstream logic keyed on “number of registrants.”

Recommendations:

1. Enforce uniqueness on-chain if the invariant must hold without trusting the operator—for example a membership PDA per (registry, wallet) with `init` on first add, or a checked scan across pages (costly at scale).
2. If uniqueness is intentionally left to the operator, consider adding documentation.

Dabba Network: Acknowledged with with [PR-4](#). Off-chain Enforcement: wallet uniqueness per role is enforced off-chain by the operator before calling `append_wallet`. On-chain scanning across paginated pages is prohibitively expensive at scale. Added documentation to `append_wallet` and `append_wallets_batch`.

[L-3] daily_mint catch-up loop can permanently brick emissions

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/mint-playground/src/lib.rs#L385](#)

Description:

daily_mint processes every missed epoch in a sequential on-chain loop:

- it recomputes yearly depreciation inside the loop
- it logs per-epoch progress
- it increments `state.total_minted` per iteration
- it updates `state.last_mint_epoch` only after the entire loop completes

If enough epochs are missed, the transaction will exceed Solana's compute budget and revert. Because `last_mint_epoch` is only written at the end, the failed call makes no progress and the backlog keeps growing.

This is a live liveness bug in the current code, but the realistic trigger is prolonged missed execution or a stale bootstrap parameter rather than a routine single missed mint. The same root cause is also reachable immediately through bad initialization parameters, because `initialize_state_and_mint` does not sanity-check `daily_mint_start_epoch` beyond storing it and setting `last_mint_epoch = daily_mint_start_epoch - 1`.

With the current one-day epoch duration, backlog accumulation is operationally slower than the tests imply. In practice, this makes the issue more likely to surface after extended downtime, a long pause window, or a badly chosen `daily_mint_start_epoch`, rather than during normal day-to-day operation.

Recommendations:

Replace the unbounded per-epoch loop with a bounded catch-up design. Two safe patterns are:

- cap the number of epochs processed per call and commit progress incrementally
- compute catch-up in larger chunks instead of iterating one epoch at a time

Also validate `daily_mint_start_epoch` during initialization so obviously stale values cannot create an immediate backlog bomb.

Dabba Network: Resolved with [PR-5](#). We have capped the `daily_mint` loop to `MAX_EPOCHS_PER_CALL` and changed `last_mint_epoch` from a jump-to-now to an incremental advance. Added Clock-based validation in `initialize_state_and_mint` rejecting `daily_mint_start_epoch` values in the past with `StartEpochTooOld` error variant

Zenith: Verified.

[L-4] New device NFT owner can delete prior-owner-linked DeviceAccount without owner continuity check

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [dabba-nft-contract/src/instructions/close_device_account.rs#L36](#)

Description:

`close_device_account` authorizes closure using current cNFT ownership (`leaf_owner` signer) plus minter signature, but does not enforce continuity with the original owner associated to stored linkage fields (for example `ho_nft_index`) in `DeviceAccount`.

This differs from the ownership model in `close_hotspot_owner_account` where only the original owner stored in the account can close.

1. A device cNFT is verified and a `DeviceAccount` is created with ownerA-linked business indices.
2. The device cNFT is transferred to a new wallet.
3. `close_device_account` is executed by minter + new current ownerB `leaf_owner`.
4. Bubblegum burn succeeds and the `DeviceAccount` is closed along with ownerA's business indices stored.

Recommendations:

1. If the intent is to ensure ownership consistency throughout the lifecycle of the device account (similar to hotspot owner account), consider persisting `leaf_owner` (or canonical owner reference) in `DeviceAccount` and enforce it in `close_device_account`.
2. If transferability should remain supported, add an explicit owner-rotation/update flow that updates stored owner linkage before close is allowed.

Dabba Network: Acknowledged with [@65db71abe5 ...](#). This is by design and not a security risk for the following reasons:

1. Minter is foundation-controlled. - The minter key is an admin role held exclusively by the Dabba foundation. No external party can co-sign a `close_device_account` transaction.
2. Device cNFTs are never transferred to end users. - Device cNFTs remain in foundation custody throughout their lifecycle. There is no scenario where an unauthorized third party becomes `leaf_owner` of a device cNFT.
3. Both required signers are the same trusted entity. - Since the foundation controls both the minter key and the `leaf_owner` wallet, the only party that can ever call `close_device_account` is the foundation itself — which constitutes a legitimate administrative action.
4. The same note has been added to the README file.

[L-5] User can prematurely burn and close hotspot owner account after protocol-funded mint

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [dabba-nft-contract/src/instructions/close_hotspot_owner_account.rs#L49](#)

Description:

`close_hotspot_owner_account` allows a registry-verified owner to burn and close while `status = UNSTAKED`. `init_and_mint_hotspot_owner` initializes every new account as `UNSTAKED`, so close is reachable immediately after mint, before any stake lifecycle. The same close path also sends close rent to a caller-supplied destination.

Close authorization does not require a prior stake lifecycle; it only requires owner signature, registry ownership verification, and current `UNSTAKED` status.

```
pub fn handler(ctx: Context<CloseHotspotOwnerAccount>) -> Result<()> {
    let hotspot_owner_account = &ctx.accounts.hotspot_owner_account;
    let asset_key = ctx.accounts.asset.key();
    let owner_key = ctx.accounts.owner.key();

    msg!("Burning Core NFT and closing HotspotOwnerAccount...");
    msg!("  Asset: {}", asset_key);
    msg!("  Owner: {}", owner_key);
    msg!(
        "    HO Address Index: {}",
        hotspot_owner_account.ho_address_index
    );

    // Verify the NFT is NOT staked (must unstake first)
    require!(
        hotspot_owner_account.status ==
        HotspotOwnerAccount::STATUS_UNSTAKED,
        DabbaNetworkError::AlreadyStaked
    );
}
```

Close rent recipient is unconstrained and may be any writable account passed by user:

```
// programs/dabba-nft-
contract/src/instructions/close_hotspot_owner_account.rs
#[account(
    mut,
    seeds = [b"hotspot_owner", asset.key().as_ref()],
    bump = hotspot_owner_account.bump,
    close = rent_destination
)]
pub hotspot_owner_account: Account<'info, HotspotOwnerAccount>,
...
#[account(mut)]
pub rent_destination: AccountInfo<'info>,
```

Impacts:

- Business logic and lifecycle impact: the protocol cannot enforce "stake at least once before close" from this instruction, so mint -> close is a valid path and can bypass the intended onboarding flow. Systems that infer lifecycle from close eligibility or burn events may misclassify never-staked exits
- Economic impact is secondary and small per account, unless other client layer vulnerabilities align and allow a phishing-style attack that crafts close txs that route rent at a larger scale.

Recommendations:

1. Encode explicit close policy on-chain: decide whether close is allowed for never-staked accounts, and enforce that policy with a dedicated initialized state. (e.g. instead of initializing status tounstaked, it can be initialized to a default state)
2. Consider constraining close rent routing to deterministic recipients (owner or treasury/minter), instead of leaving the destination fully caller-defined.

Dabba Network: Acknowledged with [@8613eb5965 ...](#). This is accepted by design. The close instruction is an owner-initiated burn, and the authorization checks (owner signer + registry CPI) are sufficient. There is no security risk:

1. Only the registry-verified owner can close.
2. The NFT is burned (destroyed), not transferred.
3. Rent recovered is ~0.001 SOL per PDA
4. rent_destination is constrained to admin_config.minter.

Rent Destination Constraint: Both close_hotspot_owner_account and close_device_account require rent_destination to be admin_config.minter. This prevents

callers from routing PDA rent refunds to arbitrary addresses. Passing any other account will fail with `InvalidRentDestination`.

[L-6] Update device field instructions accept unconstrained registry indices, allowing minter to silently re-attribute devices to arbitrary parties

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [dabba-nft-contract/src/instructions/update_device_fields.rs#L43](#)

Description:

`verify_device_collection` validates every registry index it stores into `DeviceAccount` — at minimum calling `verify_registry_index` to confirm the index is in-bounds and belongs to the correct role. The four update instructions that can overwrite those same fields — `update_lco_nft_index`, `update_ho_nft_index`, `update_bp_address_index`, `update_hw_address_index` — drop this check entirely. Any `u32` value the minter supplies is written directly, with no guard against out-of-bounds indices, wrong-role indices, or indices that belong to unrelated parties.

Each handler writes `new_value` without validation:

```
// programs/dabba-nft-contract/src/instructions/update_device_fields.rs
pub fn update_lco_nft_index(ctx: Context<UpdateDeviceField>, new_value:
    u32) -> Result<()> {
    ▷ ctx.accounts.device_account.lco_nft_index = new_value;
    ...
}
```

Impacts:

- A compromised or erring minter can write any `u32` into `lco_nft_index`, `ho_nft_index`, `bp_address_index`, or `hw_address_index`, silently re-attributing a device to an out-of-bounds slot, a wrong-role entry, or an unrelated registered party.
- Downstream systems and off-chain reward or attribution logic that treat `DeviceAccount` indices as the authoritative source of device-to-party linkage will compute incorrect attributions. The impact scales with the number of devices affected and the business logic consuming these indices.

Recommendations:

Add the matching `verify_registry_index` call in each update handler, requiring the new index to be in-bounds and in the correct role — the same check already performed at `verify_device_collection` time. Pass the appropriate registry config and wallet page accounts through the `UpdateDeviceField` context.

Dabba Network: Resolved with [@8c8117c756...](#)

Zenith: Verified.

[L-7] Staking reward accounting uses u64 and can overflow

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/dabba_apy_staking/src/state/global_state.rs#L26](#)
- [programs/dabba_apy_staking/src/state/global_state.rs#L27](#)
- [programs/dabba_apy_staking/src/state/user_stake.rs#L42](#)

Description:

The staking program tracks multiple lifetime reward quantities in u64 even though rewards can be recycled through the system many times over the protocol lifetime.

The live overflow surfaces are:

- `global.total_rewards_deposited`
- `global.total_rewards_claimed`
- `user.accumulated_rewards`, which is fed by `settle_rewards` after converting a u128 pending amount back down to u64

With a 9-decimal token, the nominal max supply is already more than half of `u64::MAX` in base units. Once rewards are recycled, lifetime deposited/claimed totals can exceed the counter width even though the live token supply never does.

The consequences differ by counter:

- `total_rewards_deposited` overflow bricks `update_daily_rewards`
- `total_rewards_claimed` overflow bricks `complete_*_unstake`
- `accumulated_rewards` overflow bricks `settle_rewards`, which blocks exit initiation for the affected position

Recommendations:

Widen cumulative and per-position reward accounting to u128, or explicitly cap / checkpoint counters so they cannot grow monotonically without bound.

Dabba Network: Resolved with [PR-9](#).

Zenith: Verified.

[L-8] daily_mint is reachable before bootstrap premint completes

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [lib.rs](#)

Description:

The program models a two-phase bootstrap: `initialize_state_and_mint` sets `pre_mint_complete = false` and leaves both treasury ATA fields at `Pubkey::default()` (`lib.rs:107-114`), then `pre_mint_tokens` performs the one-shot premint and flips the flag.

However, `daily_mint` (`lib.rs:337-366`) never checks `pre_mint_complete`. Its only prerequisite for the reward destination is `state.daily_reward_treasury_ata != Pubkey::default()`. Meanwhile, `update_daily_reward_treasury` (`lib.rs:686-700`) overwrites both `daily_reward_treasury` and `daily_reward_treasury_ata` in state without checking `pre_mint_complete`, so `config_admin` can satisfy that prerequisite before premint runs.

When premint eventually executes (`lib.rs:264-302`), it mints `min(premint_amount, max_supply - total_minted)`. Any prior daily emissions have already incremented `total_minted`, directly shrinking the premint tranche.

Scenario:

1. Call `initialize_state_and_mint` — `pre_mint_complete = false`, both ATA fields default.
2. `config_admin` calls `update_daily_reward_treasury` for a valid treasury, making `daily_reward_treasury_ata` non-default.
3. Once `daily_mint_start_epoch` is reached, `daily_mint_admin` calls `daily_mint` — succeeds.
4. `admin` later calls `pre_mint_tokens`.
5. Because `total_minted` already increased, premint mints less than `premint_amount`.

Recommendations:

Require `state.pre_mint_complete` in `daily_mint`, and reject `update_daily_reward_treasury` until premint bootstrap is complete. If pre-bootstrap treasury changes are required, record them without making `daily_mint` reachable.

Dabba Network: Resolved with [@9e98a25422 ...](#)

Zenith: Verified.

[L-9] Snapshot-based reward distribution diverges from documented Synthetix rewardPerToken accumulator model

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [dabba_apy_staking/src/instructions/update_daily_rewards.rs#L21-L25](#)

Description:

Based on documentation, the staking contract's reward mechanism is described as the "Synthetix RewardPerToken accumulator." While Dabba adopts the accumulator data structure (reward_per_token_stored, reward_per_token_paid, delta-based settlement), it does not implement continuous time-streamed reward accounting. Reward share is determined by a snapshot at deposit time, not by time staked within the period - reward distribution is vulnerable to ordering of operator staking within a time period.

Synthetix streams rewards per second via rewardRate. RPT grows continuously, and reward share is proportional to stake $\times$ time_active.

[A staker who joins mid-period earns only for their remaining time.](#)

Dabba bumps RPT in a single atomic step on each update_daily_rewards call, using the instantaneous total_weighted_stake as the denominator. Reward share is proportional to stake at the deposit moment only — time within the period is not a factor.

```
let rpt_increment = (amount as u128)
    .checked_mul(PRECISION)
    .ok_or(StakingError::MathOverflow)?
    .checked_div(global.total_weighted_stake)
    .ok_or(StakingError::MathOverflow)?;
```

Alice stakes day 0, Bob stakes at 23:59, deposit of 200 tokens arrives at 24:00:

- Synthetix: rewardRate = 200/86400 tokens/sec. Alice earns for 1440 min, Bob for 1 min.
 - Alice: ~199.93 tokens. Bob: ~0.07 tokens.
- Dabba: total_weighted_stake = 200, rpt_increment = 200/200 = 1.
 - Alice: $100 \times 1 = 100$ tokens. Bob: $100 \times 1 = 100$ tokens.

- Bob staked 1 minute but earns the same as Alice who staked 24 hours.

Recommendations:

1. Consider adding a minimum cadence guard to `update_daily_rewards` to protect against cadence drift, and document the intended operator sequencing — specifically that new stakes should be processed after the daily reward deposit.
2. Since staking is operator-controlled, ensure off-chain operations sequence the daily reward deposit before processing new stakes. This way new positions do not dilute the current deposit — they begin earning only from the next one.

Dabba Network: Acknowledged. We accept this trade-off for the simplicity and gas efficiency it provides, given our operational model:

- 24-hour reward epochs in PROD — The maximum "unfair" window is under 1 day, which is acceptable for our use case
- Operator-controlled staking — All stake operations are permissioned, preventing external actors from gaming deposit timing
- Lock periods (6—12 months) — Prevents any rapid stake/claim/unstake exploitation
- Our off-chain operator sequences transactions such that new stakes are processed before calling `update_daily_rewards`, ensuring new positions are included in the reward denominator and existing stakers are not diluted.

[L-10] mint_partner_cnft does not pin partner_collection to admin_config.partner_collection_mint

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/dabba-nft-contract/src/instructions/mint_partner_cnft.rs#L138](https://github.com/dabba-nft-contract/src/instructions/mint_partner_cnft.rs#L138)

Description:

verify_partner_collection constrains partner_collection.key() = admin_config.partner_collection_mint, so verification and collection CPI accounts are aligned with config. mint_partner_cnft accepts partner_collection as an unchecked AccountInfo and embeds ctx.accounts.partner_collection.key() into Bubblegum MetadataArgs.collection. The merkle tree is correctly checked against admin_config.partner_merkle_tree, but the collection namespace in the leaf hash can be any pubkey the minter supplies.

A minter can therefore mint partner cNFTs into the protocol's partner tree with metadata pointing at an arbitrary collection. Recipients expecting to use verify_partner_cnft_collection will fail proof verification when metadata was built with a different collection key than partner_collection_mint.

```
// programs/dabba-nft-contract/src/instructions/mint_partner_cnft.rs
/// Partner collection mint address
/// CHECK: Passed by admin, verified as collection
pub partner_collection: AccountInfo<'info>,
```

```
// programs/dabba-nft-contract/src/instructions/mint_partner_cnft.rs
(handler)
    collection: Some(Collection {
        verified: false,
        key: ctx.accounts.partner_collection.key(),
    }),
```

Recommendations:

On `MintPartnerCnft`, add the same constraint as `VerifyPartnerCollection`:
`partner_collection.key() = admin_config.partner_collection_mint.`

Dabba Network: Fixed in [b5737c9abcc255ea4a45c2b9588859dc5e9d13fb](#)

Zenith: Verified.

[L-11] Stale dabba_nft_index persists after unstake

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [programs/dabba-nft-contract/src/instructions/unstake.rs#L128-L129](#)

Description:

After unstake, the program resets status to UNSTAKED and staked_at to 0, but it does not reset dabba_nft_index. init_and_mint_hotspot_owner initializes dabba_nft_index to 0, and stake sets it to the active link value. This creates an asymmetry where an unstaked account can still carry the last linked index.

```
// programs/dabba-nft-contract/src/instructions/unstake.rs
hotspot_owner_account.status = HotspotOwnerAccount::STATUS_UNSTAKED;
hotspot_owner_account.staked_at = 0;
//@audit `dabba_nft_index` is left unchanged, so the last linked value
persists.
```

Impacts: Off-chain indexers or backends that treat non-zero dabba_nft_index as "actively linked" may report stale associations after unstake if they do not also gate on status.

Recommendations:

Either "unstaked means no active link" (reset dabba_nft_index to 0 on unstake) or "preserve last linked index for history" (document and ensure all consumers key off status and/or events).

Dabba Network: Resolved with [@d2bdbf6ef4 ...](#)

Zenith: Verified.

[L-12] Init allows Pubkey::default() for super admin and role authority while transfers reject it

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/registry_contract/src/lib.rs#L89](#)
- [programs/registry_contract/src/lib.rs#L147](#)

Description:

`initialize_global_config` and `initialize_registry` accept arbitrary Pubkey arguments and persist `super_admin` and `authority` without rejecting the all-zero public key (`Pubkey::default()`). The same sentinel is explicitly forbidden on `transfer_super_admin` and `transfer_authority`, so init and transfer handling are inconsistent. This is not a typical end-user exploit: only the party that successfully runs each init (global config once per deployment; each registry once per role) supplies those values, so the main failure mode is misconfiguration or unsafe deployment scripting.

`initialize_global_config` stores `super_admin` with no $\neq$ `Pubkey::default()` guard:

```
// programs/registry_contract/src/lib.rs
pub fn initialize_global_config(
    ctx: Context<InitializeGlobalConfig>,
    super_admin: Pubkey,
) -> Result<()> {
    let config = &mut ctx.accounts.global_config;
    config.super_admin = super_admin;
    config.paused = false;
    config.bump = ctx.bumps.global_config;
```

`initialize_registry` stores `authority` with no matching guard:

```
// programs/registry_contract/src/lib.rs
registry.version = 1;
registry.role = role;
    registry.authority = authority;
    registry.total_wallets = 0;
```

By contrast, transfers already reject the default pubkey:

```
// programs/registry_contract/src/lib.rs
require!(
    new_super_admin ≠ Pubkey::default(),
    ErrorCode::InvalidAuthority
);
```

```
// programs/registry_contract/src/lib.rs
require!(
    new_authority ≠ Pubkey::default(),
    ErrorCode::InvalidAuthority
);
```

Recommendations:

1. In `initialize_global_config`, add `require!(super_admin ≠ Pubkey::default(), ErrorCode::InvalidAuthority)` (or a dedicated error) before writing `config.super_admin`.
2. In `initialize_registry`, add the same class of check for authority before writing `registry.authority`, reusing `InvalidAuthority` or a dedicated init error for clarity.

Dabba Network: Resolved with [@38c5d73b5a ...](#)

Zenith: Verified.

4.4 Informational

A total of 12 informational findings were identified.

[I-1] Pause scope is inconsistent across docs; Emergency pause omits super-admin rotation

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/registry_contract/src/lib.rs#L105-L113](#)

Description:

Documentation diverges on pause behavior. Documentation states an invariant that pausing blocks all mutations except unpause. The inline comment on pause says it blocks append and transfer operations, which reads like role-registry updates, not an explicit list of every super-admin-only instruction. Implementation matches the narrower reading for append and role authority transfer, but not the readme's global "all mutations except unpause" wording.

```
// programs/registry_contract/src/lib.rs
pub fn transfer_super_admin(
    ctx: Context<TransferSuperAdmin>,
    new_super_admin: Pubkey,
) -> Result<> {
    let config = &mut ctx.accounts.global_config;
    require!(
        new_super_admin != Pubkey::default(),
        ErrorCode::InvalidAuthority
    );
    require!(
        config.super_admin == *ctx.accounts.super_admin.key,
        ErrorCode::Unauthorized
    );
    let old_super_admin = config.super_admin;
    config.super_admin = new_super_admin;
```

```
...  
}
```

There is no `require(!(config.paused, ErrorCode::ProgramPaused))` before the state update, unlike the pause-gated `append` and `transfer_authority` paths.

Impacts: During an emergency pause, super-admin rotation can still occur on-chain if the documentation's broad invariant is assumed, so pause cannot be relied on to freeze that control plane.

Recommendations:

Consider picking one documented pause policy: either narrow the readme to match implemented scope (list what is pause-gated, explicitly exclude or include super-admin-only instructions), or add `require(!(config.paused, ErrorCode::ProgramPaused))` to every mutating instruction except `unpause`.

Dabba Network: Resolved with [PR-2](#). Added `require(!(config.paused, ErrorCode::ProgramPaused))` to `transfer_super_admin`. All mutating instructions are now pause-gated except `unpause`.

Zenith: Verified.

[I-2] Incorrect error for reward-vault init ordering failure

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/dabba_apy_staking/src/instructions/initialize_vaults.rs#L37](#)

Description:

handler_reward_vault requires stake-vault initialization first, but reports InvalidMint when ordering is wrong.

```
// programs/dabba_apy_staking/src/instructions/initialize_vaults.rs
// Stake vault must be initialized first
require!(
    global.stake_vault != Pubkey::default(),
    StakingError::InvalidMint
);
```

```
// programs/dabba_apy_staking/src/errors.rs
#[msg("Token account mint does not match stake mint")]
InvalidMint,
```

Recommendations:

Add a dedicated error variant for wrong initialization order or uninitialized stake vault

Dabba Network: Resolved with [PR-3](#)

Zenith: Verified.

[I-3] Unused StakingError variants LockExpired and PositionCooling

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/dabba_apy_staking/src/errors.rs#L30](#)
- [programs/dabba_apy_staking/src/errors.rs#L72](#)

Description:

LockExpired and PositionCooling are declared on StakingError but no instruction returns or uses them.

```
// programs/dabba_apy_staking/src/errors.rs
#[msg("Lock period has expired – cannot add to this position")]
▷ LockExpired,

...

#[msg("Cannot stake into a position that is in cooling phase")]
▷ PositionCooling,
```

Recommendations:

Consider removing LockExpired and PositionCooling from StakingError if unused. If these are planned to be used in the future, consider adding short comments above each variant stating they are reserved.

Dabba Network: Resolved with [@e406b999f4 ...](#)

Zenith: Verified.

[I-4] Unused token metadata constants DBT_DECIMALS and DBT_UNIT

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/dabba_apy_staking/src/constants.rs#L2-L6](#)

Description:

DBT_DECIMALS and DBT_UNIT are defined for documentation of a 9-decimal token, but the program does not reference them in staking, reward, or vault logic.

```
// programs/dabba_apy_staking/src/constants.rs
/// DBT token decimals
▷ pub const DBT_DECIMALS: u8 = 9;

/// 1 DBT in raw base units (10^9)
#[allow(dead_code)]
▷ pub const DBT_UNIT: u64 = 1_000_000_000;
```

Recommendations:

1. If the product invariant is “stake mint must be 9 decimals,” add an explicit check (for example in `initialize` when `stake_mint` is first stored) comparing `ctx.accounts.stake_mint.decimals` to `DBT_DECIMALS`
2. Or consider removing `DBT_DECIMALS` and `DBT_UNIT`.

Dabba Network: Resolved with [@d2e197649b ...](#)

Zenith: Verified.

[I-5] Manual stake allows LockPeriod::Genesis, diverging from 6m/12m-only intent

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/dabba_apy_staking/src/instructions/stake_manual.rs#L33](#)

Description:

Current documentation describes two staking paths: `genesis_stake` for hotspot reward positions (fixed 200-day lock, 1.5x, `genesis_stake` PDA seeds), and `stake_manual` for direct DBT with lock choice limited to 6 and 12 months. The on-chain `stake_manual` handler does not restrict `lock_period` to those tiers and allow `genesis` tier.

```
// programs/dabba_apy_staking/src/instructions/stake_manual.rs
pub fn handler(
    ctx: Context<StakeManual>,
    nonce: u32,
    amount: u64,
    lock_period: LockPeriod,
) -> Result<()> {
    ...
    ▷ user.lock_period = lock_period;
    user.lock_start_ts = clock.unix_timestamp;
    user.lock_end_ts = clock
        .unix_timestamp
        .checked_add(GlobalStakingState::lock_duration(&lock_period))
        .ok_or(StakingError::MathOverflow)?;
    ▷ user.lock_multiplier_bps = global.multiplier_for(&lock_period);
    ...
}
```

Impacts:

- Access remains operator-only (custodial model per audit scope), so this is a documentation and code-consistency issue.
- Off-chain indexing, reporting, or invariants that assume manual positions only use lock

bytes 0 and 1 may miscount or mislabel Genesis manual positions.

Recommendations:

Align on-chain behavior with README intent: reject `LockPeriod::Genesis` in `stake_manual` so hotspot genesis positions use only `genesis_stake` and manual PDAs only represent 6m/12m tiers.

Dabba Network: Resolved with [@e42a932c7a ...](#)

Zenith: Verified.

[I-6] Unused recompute_weight

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/dabba_apy_staking/src/state/user_stake.rs#L100](#)

Description:

UserStakeAccount::recompute_weight is present but unused, while update_lock_multipliers updates only global defaults for future stakes.

```
pub fn recompute_weight(&mut self) -> Result<(u128, u128)> {  
    let old_weight = self.weighted_stake;  
  
    let new_weight = (self.staked_amount as u128)  
        .checked_mul(self.lock_multiplier_bps as u128)  
        .ok_or(StakingError::MathOverflow)?  
        .checked_div(MULTIPLIER_DENOMINATOR)  
        .ok_or(StakingError::MathOverflow)?;  
  
    self.weighted_stake = new_weight;  
  
    Ok((old_weight, new_weight))  
}
```

Recommendations:

1. Consider removing recompute_weight if not needed.
2. Document explicit policy: multiplier updates affect only new positions, not existing ones.

Dabba Network: Resolved with [@651f293ab7 ...](#)

Zenith: Verified.

[I-7] Insufficient validations

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/dabba-nft-contract/src/instructions/verify_device_collection.rs#L381](#)
- [programs/dabba-nft-contract/src/instructions/verify_partner_collection.rs#L79](#)
- [programs/dabba-nft-contract/src/instructions/stake.rs#L16](#)

Description:

Three related defense-in-depth gaps: local checks differ across flows where a stricter pattern exists elsewhere.

1. Location owner index not registry-checked. `verify_device_collection` calls `verify_registry_index` for LCO, BP, and HW but stores `location_owner_index` from instruction data without `verify_registry_index`.

```
// programs/dabba-nft-contract/src/instructions/verify_device_collection.rs
// ...
device.location_owner_index = location_owner_index;
```

2. PDA not checked locally:

`verify_partner_collection`:

- `tree_config`
- `collection_metadata`
- `collection_edition`
- `bubblegum_signer`

`mint_device_cnft`, `mint_partner_cnft`:

- `tree_config`

3. Stake and unstake omit `admin_config.is_initialized` on the PDA constraint; other instructions require both initialized state and not paused.

Recommendations:

1. Device verify: add location-owner registry accounts and `verify_registry_index`
2. Align `verify_partner_collection` with `verify_device_collection` PDA checks for tree config, collection metadata/edition, and Bubblegum signer.
3. Add `admin_config.is_initialized` to Stake and Unstake to match the rest of the program.

Dabba Network: Resolved with [@905deef5fa ...](#)

Zenith: Verified.

[I-8] Hotspot-owner registry slots are not uniquely bound to assets

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [programs/dabba-nft-contract/src/instructions/init_and_mint_hotspot_owner.rs#L13](#)
- [programs/dabba-nft-contract/src/instructions/init_and_mint_hotspot_owner.rs#L98-L104](#)

Description:

`init_and_mint_hotspot_owner` verifies that a wallet is registered at `ho_address_index`, but it does not ensure that this registry slot has already been bound to a different `HotspotOwner` asset.

The PDA for `HotspotOwnerAccount` is keyed only by the new asset address, so the minter can repeat the flow with different asset signers and mint multiple hotspot-owner NFTs that all point at the same registry slot.

Later lifecycle instructions continue to authenticate each asset independently by consulting the stored `ho_address_index`, so the duplicates remain live rather than collapsing onto a single canonical record.

Only the configured minter role (a trusted backend service per `AdminConfig`) can call this instruction — it is not permissionless. Exploitation therefore requires the backend to be buggy or compromised; no external user can trigger duplicate minting directly. The duplicates do not drain funds but could create inconsistent on-chain state for the affected hotspot-owner identity.

Recommendations:

Add a uniqueness record keyed by the registry slot being claimed, such as a PDA derived from `ho_address_index` or `ho_nft_index`, and require that it is initialized only once per active hotspot-owner identity.

Dabba Network: Acknowledged. A single hotspot owner can operate multiple hotspot devices. Each device gets its own HO NFT, all pointing to the same `ho_address_index` (the owner's registry slot). The PDA is keyed by asset address, not registry index, intentionally allowing multiple NFTs per owner. Only the trusted minter can call this instruction —

duplicate prevention is handled at the backend/business logic layer.

[I-9] Delayed daily-mint backlog always follows the latest configured reward treasury

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- lib.rs

Description:

State ([lib.rs:976-985](#)) tracks only one active `daily_reward_treasury`, one `daily_reward_treasury_ata`, and one `last_mint_epoch` — there is no per-epoch routing history.

`daily_mint` ([lib.rs:368-375](#)) derives backlog strictly from `current_epoch - last_mint_epoch`. All missed epochs are summed into one `total_amount_minted` and delivered through a single `mint_to` CPI targeting the current `daily_reward_treasury_ata` ([lib.rs:438-475](#)).

`update_daily_reward_treasury` ([lib.rs:696-700](#)) overwrites both the treasury wallet and its ATA in place with no checkpointing for previously missed epochs.

This means a treasury rotation retroactively redirects all unsettled backlog to the new destination.

Real-world impact is minimal: `config_admin` (treasury rotator) and `daily_mint_admin` (mint caller) are both trusted admin roles, and both the old and new treasuries are protocol-controlled wallets. No user or external funds are at risk — only freshly minted protocol reward tokens are redirected between admin-owned wallets.

Scenario:

1. Initialize the program and complete premint.
2. Let several mint epochs pass without calling `daily_mint`.
3. `config_admin` calls `update_daily_reward_treasury` to point state at a new treasury.
4. `daily_mint_admin` calls `daily_mint`.
5. The handler computes the full backlog from the stale `last_mint_epoch`, aggregates all missed epochs, and mints the entire amount to the new treasury ATA.

Recommendations:

Require backlog settlement before treasury rotation, or persist routing checkpoints so missed epochs remain tied to the treasury that was active when those epochs elapsed.

Dabba Network: Acknowledged. Both `daily_reward_treasury` (old and new) are protocol-controlled admin wallets. The backlog going to the new treasury after rotation is the intended behavior — operationally, the admin would call `daily_mint` to settle any pending epochs before rotating the treasury. Additionally, with the `pre_mint_complete` gate on `update_daily_reward_treasury` fix, treasury rotation is further restricted to only occur after the bootstrap phase is complete. Since both treasuries are admin-owned and no user or external funds are at risk, we consider this acceptable protocol behavior and not a security concern.

[I-10] Initialize allows default role addresses for critical authorities

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/dabba_apy_staking/src/instructions/initialize.rs#L46-L48](#)

Description:

`initialize` writes `reward_depositor`, `operator`, and `pause_authority` without rejecting `Pubkey::default()`. Later role-update instructions enforce non-default addresses, so initialization has weaker validation than steady-state governance.

```
global.reward_depositor = reward_depositor;
global.operator = operator;
global.pause_authority = pause_authority;
```

Impacts:

- Misconfigured deployment can soft-brick operator or reward flows until super-admin fixes roles.
- Initialization behavior is inconsistent with post-deploy role hygiene, increasing operator error risk.

Recommendations:

Add explicit non-default checks in `initialize` for all role addresses. For example,

```
require!(reward_depositor != Pubkey::default(),
  StakingError::InvalidRoleAddress);
require!(operator != Pubkey::default(), StakingError::InvalidRoleAddress);
require!(pause_authority != Pubkey::default(),
  StakingError::InvalidRoleAddress);
```

Dabba Network: Resolved with [@b84afa4c6f...](#)

Zenith: Verified.

[I-11] Unused TooManyYearsPassed error variant creates dead error path ambiguity

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/mint-playground/src/lib.rs#L1112](#)

Description:

A custom error variant is declared but never used by any validation or execution path.

```
// programs/mint-playground/src/lib.rs
#[error_code]
pub enum CustomError {
    // ...
    TooManyYearsPassed,
    // ...
}
```

This is a defensive/code-quality issue rather than an exploitable vulnerability, but it can mislead reviewers and future maintainers about expected guardrails in the mint logic.

Recommendations:

1. Remove TooManyYearsPassed if no upper-bound check is intended.
2. If a bound is intended, add the corresponding runtime guard in the relevant mint/depreciation logic and return CustomError::TooManyYearsPassed.

Dabba Network: Resolve with [@c30354001f ...](#)

Zenith: Verified.

[I-12] Redundant treasury_ata check after writing state.treasury_ata

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/mint-playground/src/lib.rs#L227-L228](#)

Description:

In `pre_mint_tokens`, `state.treasury_ata` is assigned from `ctx.accounts.treasury_ata.key()`, then a `require_keys_eq!` compares those two values. That comparison cannot fail under normal execution. Real binding of the treasury ATA to protocol state is already enforced earlier via `treasury` vs `state.treasury` and ATA address derivation against `get_associated_token_address`.

```
// programs/mint-playground/src/lib.rs
state.treasury_ata = ctx.accounts.treasury_ata.key();
state.daily_reward_treasury_ata
= ctx.accounts.daily_reward_treasury_ata.key();
...
require_keys_eq!(
  ctx.accounts.treasury_ata.key(),
  state.treasury_ata,
  CustomError::InvalidTreasury
);
```

Recommendations:

Consider removing the redundant `require_keys_eq!` block that compares `ctx.accounts.treasury_ata` to `state.treasury_ata` after the assignment.

Dabba Network: Resolved with [PR-2](#). Removed the duplicate ATA derivation check and the self-comparison `require_keys_eq!` block. The real validation is already performed earlier in the function via `get_associated_token_address`.

Zenith: Verified.